React JS

Concise study notes: fundamentals, hooks, routing and performance

1. Fundamentals

What is React?

React is a JavaScript library for building user interfaces, maintained by Meta and the open-source community. It is **declarative** (you describe what the UI should look like for a given state), **component-based** (UIs are built from small, reusable pieces) and follows **unidirectional data flow** (data moves from parent to child via props).

Feature	What it means
Virtual DOM	An in-memory tree of the UI. On every update React builds a new tree, diffs it against the old one (reconciliation) and applies only the minimal real DOM changes.
Components	Independent, reusable functions that return UI (JSX).
JSX	Syntax extension that looks like HTML but compiles to `React.createElement` / jsx() calls.
One-way binding	Parent passes data down; children notify parents through callback props.
Ecosystem	React Router, Redux Toolkit, Zustand, TanStack Query, Next.js, Vite, etc.

Setting up a project

```
npm create vite@latest my-app -- --template react-ts
cd my-app
npm install
npm run dev          # dev server at http://localhost:5173
```

JSX rules

- A component must return **one root element**; use a Fragment `<>...</>` to avoid extra DOM nodes.
- Use `className` instead of class, `htmlFor` instead of for, and camelCase for attributes (onClick, tabIndex).
- Embed JavaScript expressions with curly braces: `{user.name}`. Statements (if, for) are not allowed inside braces.
- Every tag must be closed, including `<img />` and `<input />`.
- Inline styles take an object: `style={{ color: 'red', fontSize: 14 }}`.

```
function Welcome() {
  const user = { name: "Shiv", role: "Developer" };
  return (
    <>
      <h1 className="title">Hello, {user.name}!</h1>
      <p>Role: {user.role.toUpperCase()}</p>
    </>
  );
}
```

Components and props

A component is a function that takes **props** (read-only inputs) and returns JSX. Props must never be mutated. The special `children` prop holds whatever is nested between the opening and closing tags.

```
function Card({ title, children, onClose }) {
  return (
    <div className="card">
      <h3>{title}</h3>
      {children}
      <button onClick={onClose}>Close</button>
    </div>
  );
}

<Card title="Profile" onClose={() => setOpen(false)}>
  <p>Some content here</p>
</Card>
```

2. Rendering, State and Events

Conditional rendering

```
{isLoggedIn ? <Dashboard /> : <Login />}      // ternary
{items.length > 0 && <List items={items} />}  // logical AND
if (loading) return <Spinner />;               // early return
```

Gotcha: `{count && <X />}` renders the number 0 when count is 0. Use `count > 0 && ...` instead.

Lists and keys

Render arrays with `map()`. Each item needs a stable, unique **key** so React can track it between renders. Use a database id, not the array index, whenever items can be reordered, inserted or removed.

```
<ul>
  {todos.map(todo => (
    <li key={todo.id}>{todo.text}</li>
  ))}
</ul>
```

State with useState

State is data a component remembers between renders. Calling the setter schedules a re-render. State updates are **asynchronous and batched**, so use the functional form when the new value depends on the old one.

```
const [count, setCount] = useState(0);

setCount(count + 1);          // uses value from this render
setCount(prev => prev + 1);   // safe: always uses latest value

// Objects and arrays: never mutate, always create a copy
setUser(prev => ({ ...prev, name: "Aman" }));
setTodos(prev => [...prev, newTodo]);
setTodos(prev => prev.filter(t => t.id !== id));
```

Handling events and forms

Events are camelCase and receive a SyntheticEvent. In a **controlled component** the input value lives in React state, making state the single source of truth.

```
function LoginForm() {
  const [form, setForm] = useState({ email: "", password: "" });

  const handleChange = e =>
    setForm(prev => ({ ...prev, [e.target.name]: e.target.value }));

  const handleSubmit = e => {
    e.preventDefault();
    console.log(form);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input name="email" value={form.email} onChange={handleChange} />
      <input name="password" type="password"
             value={form.password} onChange={handleChange} />
      <button type="submit">Login</button>
    </form>
  );
}
```

Lifting state up

When two sibling components need the same data, move the state to their closest common parent and pass it down as props (along with setter callbacks). If this gets deeply nested (prop drilling), reach for Context or a state library.

3. Hooks

Hooks let function components use state, lifecycle and other React features. **Rules of Hooks:** call them only at the top level (never inside loops, conditions or nested functions) and only from components or other custom hooks.

useEffect: side effects

Runs *after* render. Use it to synchronise with external systems: data fetching, subscriptions, timers, DOM APIs. The dependency array controls when it re-runs, and the returned cleanup function runs before the next effect and on unmount.

Dependency array	When the effect runs
`useEffect(fn)`	After every render
`useEffect(fn, [])`	Once, after the first render (mount)
`useEffect(fn, [a, b])`	On mount and whenever a or b changes

```
useEffect(() => {
  const controller = new AbortController();

  fetch(`/api/users/${id}`, { signal: controller.signal })
    .then(res => res.json())
    .then(setUser)
    .catch(err => { if (err.name !== "AbortError") setError(err); });

  return () => controller.abort();   // cleanup
}, [id]);
```

> **Tip:** you often do not need an effect. Derive values during render, handle user actions in event handlers, and use a data library (TanStack Query / SWR) for server state.

Other core hooks

Hook	Purpose	Example
useRef	Mutable value that survives renders without causing re-render; access DOM nodes.	`const inputRef = useRef(null); inputRef.current.focus()`
useContext	Read a value from the nearest Context provider; avoids prop drilling.	`const theme = useContext(ThemeContext)`
useReducer	Complex state logic with a reducer function (like a mini Redux).	`const [state, dispatch] = useReducer(reducer, init)`
useMemo	Cache the result of an expensive calculation.	`const sorted = useMemo(() => sort(list), [list])`
useCallback	Cache a function reference between renders.	`const onSave = useCallback(() => save(id), [id])`
useLayoutEffect	Like useEffect but fires synchronously before paint (measure layout).	Rarely needed
useId / useTransition	Stable ids for accessibility; mark updates as non-urgent.	React 18+

Context API

Context shares values (theme, auth user, locale) across the tree without passing props at every level. Every consumer re-renders when the provided value changes, so keep context values small and split contexts by concern.

```
const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const value = useMemo(() => ({ user, setUser }), [user]);
  return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
}

export const useAuth = () => useContext(AuthContext);
```

Custom hooks

A custom hook is a function whose name starts with `use` and which calls other hooks. It lets you extract and reuse stateful logic (not UI) across components.

```
function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    let ignore = false;
    setLoading(true);
    fetch(url)
      .then(r => r.json())
      .then(d => { if (!ignore) setData(d); })
      .catch(e => { if (!ignore) setError(e); })
      .finally(() => { if (!ignore) setLoading(false); });
    return () => { ignore = true; };
  }, [url]);

  return { data, loading, error };
}

// usage
const { data: users, loading } = useFetch("/api/users");
```

4. Routing, Performance and Lifecycle

React Router (v6+)

Client-side routing for single-page apps. Navigation swaps components without a full page reload.

```
import { BrowserRouter, Routes, Route, Link,
         useParams, useNavigate, Navigate } from "react-router-dom";

function App() {
  return (
    <BrowserRouter>
      <nav><Link to="/">Home</Link> <Link to="/users">Users</Link></nav>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/users/:id" element={<UserDetail />} />
        <Route path="/old" element={<Navigate to="/" replace />} />
        <Route path="*" element={<NotFound />} />
      </Routes>
    </BrowserRouter>
  );
}

function UserDetail() {
  const { id } = useParams();
  const navigate = useNavigate();
  return <button onClick={() => navigate(-1)}>Back (user {id})</button>;
}
```

Performance optimisation

- **React.memo(Component)**: skip re-rendering when props are shallow-equal. Pair with useCallback/useMemo for function and object props.
- **Code splitting:** `const Admin = lazy(() => import('./Admin'))` wrapped in `<Suspense fallback={...}>`.
- **Virtualise long lists** with react-window or TanStack Virtual instead of rendering thousands of rows.
- **Keep state local:** state placed too high re-renders large subtrees needlessly.
- **Stable keys** and avoiding inline object/array creation in props that feed memoised children.
- **Measure first** with React DevTools Profiler; do not memoise everything by default.
- **React Compiler** (React 19 era) can auto-memoise many components, reducing manual useMemo/useCallback.

```
const Chart = lazy(() => import("./Chart"));

<Suspense fallback={<p>Loading...</p>}>
  <Chart data={data} />
</Suspense>
```

Component lifecycle (class vs hooks)

Phase	Class component	Function component
Mount	componentDidMount	`useEffect(() => {...}, [])`
Update	componentDidUpdate	`useEffect(() => {...}, [deps])`
Unmount	componentWillUnmount	Cleanup function returned from useEffect
Error handling	componentDidCatch / getDerivedStateFromError	Error Boundary (still a class, or react-error-boundary)

Error boundaries

A component that catches render errors in its child tree and shows a fallback UI instead of crashing the whole app. They do not catch errors in event handlers, async code or the boundary itself.

5. Best Practices and Quick Revision

Best practices

- Keep components small and focused; extract when a component does more than one job.
- Never mutate state or props directly; always return new objects and arrays.
- Use meaningful, stable **keys**; avoid index keys for dynamic lists.
- Colocate state with where it is used; lift it only as far as necessary.
- Prefer derived values over duplicated state (compute *filteredList* from list + query during render).
- Always clean up effects (listeners, timers, subscriptions, aborted requests).
- Use TypeScript for props and API types; it catches a large class of bugs early.
- Handle loading, error and empty states explicitly in every data-driven view.
- Use semantic HTML and accessible attributes (labels, aria-*, keyboard support).
- Lint with eslint-plugin-react-hooks to enforce the Rules of Hooks and dependency arrays.

Common interview questions (one-line answers)

Question	Answer
Virtual DOM vs real DOM?	Virtual DOM is a lightweight JS copy; React diffs it and patches only changed real DOM nodes.
State vs props?	Props are read-only inputs from a parent; state is private, mutable data owned by the component.
Controlled vs uncontrolled?	Controlled: value driven by React state. Uncontrolled: DOM holds the value, read via ref.
Why keys in lists?	They give each item a stable identity so reconciliation can reorder/update correctly.
useMemo vs useCallback?	useMemo caches a computed value; useCallback caches a function reference.
What is prop drilling?	Passing props through many layers that do not use them; solved with Context or state libraries.
useEffect vs useLayoutEffect?	useEffect runs after paint; useLayoutEffect runs before paint, blocking it.
Why can't hooks be conditional?	React relies on call order to match hook state between renders.

Hooks cheat sheet

```
const [x, setX] = useState(init);                 // local state
useEffect(() => { ...; return cleanup; }, [deps]); // side effects
const ref = useRef(null);                          // DOM / mutable box
const val = useContext(Ctx);                       // read context
const [s, dispatch] = useReducer(fn, init);        // complex state
const memo = useMemo(() => compute(a), [a]);       // cached value
const cb = useCallback(() => doIt(a), [a]);        // cached function
```

Next steps: build a small project (todo app, weather app, or a dashboard with auth + routing), then explore Next.js, TanStack Query for server state, and Redux Toolkit or Zustand for global state. Official docs: react.dev.